

lab05_Antunes

AUTHOR

Danilo Antunes

Lab 05

Ao longo desta semana, você deverá desenvolver código em R, utilizando-se de tidyverse, ggplot, leaflet, ggcal e outros pacotes disponibilizados em HTML Widgets para responder uma questão relacionada ao conjunto de dados flights. Para tanto, considere os seguintes fatos:

- Cias Aéreas (AIRLINE) possuem diferentes aeronaves (codificadas em TAIL_NUMBER);
- (Exemplo hipotético) No dia 01/01/2015, a aeronave N431WN viaja de BWI para JFK;
- (Exemplo hipotético) No dia 02/01/2015, a aeronave N431WN viaja de JFK para ORD';
- E assim sucessivamente. . .

No caso apontado acima, pode-se ver que a aeronave N431WN realizou um trajeto (provavelmente único) durante todo o ano de 2015. Nos dias exemplificados, o trajeto foi BWI-JFK-ORD.

Crie uma função que:

- a. Receba um valor de TAIL_NUMBER (por exemplo, N431WN);
- b. Produza uma tabela (tidy) com todos os trajetos realizados pela aeronave (ordenadas por data e hora, contendo todas as colunas do arquivo flights.csv.zip);
- c. Produza um mapa que apresente todo o trajeto voado pela aeronave ao longo de todo o ano; o trajeto deve ser apresentado de maneira linear no tempo (i.e., segue a sequência do tempo, como no exemplo hipotético dado acima);
- d. O mapa deve ser decorado com estatísticas do seu interesse (por exemplo, velocidade média do voo como espessura da linha que conecta os aeroportos envolvidos no trajeto);

Observações:

1. A função terá o argumento tail_number e arquivo;
2. O argumento tail_number será o código identificador do avião de interesse (por ex., N431WN);
3. O argumento arquivo será o nome do arquivo com caminho completo (por ex., file.path("dados", "flights.csv.zip"));
4. A função se chamará analisa_aeronave;
5. O arquivo deve ser lido por partes;
6. O mapa deve utilizar as coordenadas exatas dos aeroportos envolvidos (presentes no arquivo airports);

7. A função deve retornar uma lista (nomeada) com dois valores: a tabela utilizada para o gráfico e o gráfico propriamente dito;

```
library(tidyverse)
```

```
— Attaching core tidyverse packages — tidyverse 2.0.0 —
✓ dplyr      1.2.1      ✓ readr      2.2.0
✓ forcats    1.0.1      ✓ stringr    1.6.0
✓ ggplot2    4.0.3      ✓ tibble     3.3.1
✓ lubridate  1.9.5      ✓ tidyr      1.3.2
✓ purrr      1.2.2

— Conflicts — tidyverse_conflicts() —
✖ dplyr::filter() masks stats::filter()
✖ dplyr::lag()     masks stats::lag()
i Use the conflicted package (<http://conflicted.r-lib.org/>) to force all conflicts to become errors
```

```
library(ggplot2)
library(leaflet)
library(ggcal)

analisa_aeronave <- function(tail_number, arquivo) {

  # importação dos dados
  getStats <- function(input, pos) {
    input %>%
      filter(TAIL_NUMBER == tail_number) %>%
      filter(
        !is.na(YEAR),
        !is.na(MONTH),
        !is.na(DAY),
        !is.na(TAIL_NUMBER),
        !is.na(ORIGIN_AIRPORT),
        !is.na(DESTINATION_AIRPORT),
        !is.na(DEPARTURE_TIME)
      ) %>%
      filter(!(substr(DESTINATION_AIRPORT, 1, 1) == "1")) %>%
      arrange(YEAR, MONTH, DAY, DEPARTURE_TIME)
  }

  dados = read_csv_chunked(
    arquivo,
    callback=DataFrameCallback$new(getStats),
    chunk_size = 1e5
  )

  airports <- read_csv("datasets/airports.csv")

  # unindo os dois datasets
  dados_mapa <- dados %>%
    left_join(
      airports %>% select(IATA_CODE, LATITUDE, LONGITUDE, AIRPORT, CITY),
      ...
    )
}
```

```

    by = c("ORIGIN_AIRPORT" = "IATA_CODE")
  ) %>%
  rename(lat_orig = LATITUDE,
         lng_orig = LONGITUDE,
         nome_orig = AIRPORT,
         cidade_orig = CITY) %>%
  left_join(
    airports %>% select(IATA_CODE, LATITUDE, LONGITUDE, AIRPORT, CITY),
    by = c("DESTINATION_AIRPORT" = "IATA_CODE")
  ) %>%
  rename(lat_dest = LATITUDE,
         lng_dest = LONGITUDE,
         nome_dest = AIRPORT,
         cidade_dest = CITY) %>%
  filter(!is.na(lat_orig), !is.na(lng_orig), !is.na(lat_dest), !is.na(lng_dest)) %>%
  mutate(
    # adicionando coluna de velocidade
    AIR_TIME = as.numeric(AIR_TIME),
    DISTANCE = as.numeric(DISTANCE),
    velocidade_mph = ifelse(
      !is.na(AIR_TIME) & AIR_TIME > 0,
      DISTANCE / (AIR_TIME / 60),
      NA
    )
  )

# criando o mapa

cores <- colorNumeric(
  palette = "YlOrRd",
  domain = dados_mapa$velocidade_mph,
  na.color = "#2b5c8f"
)

mapa <- leaflet() %>%
  addProviderTiles(providers$CartoDB.Positron)

for (i in seq_len(nrow(dados_mapa))) {
  row <- dados_mapa[i, ]

  espessura <- 0.01 + (row$velocidade_mph / 100)
  cor_linha <- cores(row$velocidade_mph)

  popup_txt <- paste0(
    "<b>Trecho #</b>", i, "<br/>",
    "<b>Data:</b> ", sprintf("%02d/%02d/%04d", row$DAY, row$MONTH, row$YEAR), "<br/>",
    "<b>Voo:</b> ", row$AIRLINE, " ", row$FLIGHT_NUMBER, "<br/>",
    "<b>Trajeto:</b> ", row$ORIGIN_AIRPORT, " → ", row$DESTINATION_AIRPORT, "<br/>",
    "<b>Distância:</b> ", row$DISTANCE, " milhas<br/>",
    "<b>Velocidade Média:</b> ", paste0(round(row$velocidade_mph, 1), " mph")
  )

  mapa <- mapa %>%
    addPolyline(

```

```

      addPolyLines(
        lng = c(row$lng_orig, row$lng_dest),
        lat = c(row$lat_orig, row$lat_dest),
        weight = espessura,
        color = cor_linha,
        opacity = 0.6,
        popup = popup_txt
      )
    }

# adicionando os aeroportos visitados ao mapa
airports_visitados <- bind_rows(
  dados_mapa %>% select(IATA_CODE = ORIGIN_AIRPORT,
                        LATITUDE = lat_orig,
                        LONGITUDE = lng_orig,
                        NOME = nome_orig,
                        CIDADE = cidade_orig),
  dados_mapa %>% select(IATA_CODE = DESTINATION_AIRPORT,
                        LATITUDE = lat_dest,
                        LONGITUDE = lng_dest,
                        NOME = nome_dest,
                        CIDADE = cidade_dest)
) %>% distinct(IATA_CODE, .keep_all = TRUE)

mapa <- mapa %>%
  addCircleMarkers(
    data = airports_visitados,
    lng = ~LONGITUDE,
    lat = ~LATITUDE,
    radius = 4,
    color = "#1d3557",
    fillColor = "#457b9d",
    fillOpacity = 0.9,
    weight = 1.5,
    popup = ~paste0("<b>", IATA_CODE, "</b> - ", NOME, "<br/>", CIDADE)
  ) %>%
  addLegend(
    pal = cores,
    values = dados_mapa$velocidade_mph,
    title = "Velocidade (mph)",
    position = "bottomright"
  )

return(list(
  tabela = dados,
  mapa = mapa
))
}

```

```

arquivo <- file.path("datasets", "flights.csv.zip")

res <- analisa_aeronave("N431WN", arquivo)

```

```

— Column specification —————
cols(
  .default = col_double(),
  AIRLINE = col_character(),
  TAIL_NUMBER = col_character(),
  ORIGIN_AIRPORT = col_character(),
  DESTINATION_AIRPORT = col_character(),
  SCHEDULED_DEPARTURE = col_character(),
  DEPARTURE_TIME = col_character(),
  WHEELS_OFF = col_character(),
  WHEELS_ON = col_character(),
  SCHEDULED_ARRIVAL = col_character(),
  ARRIVAL_TIME = col_character(),
  CANCELLATION_REASON = col_character()
)
i Use `spec()` for the full column specifications.

```

Rows: 322 Columns: 7

```

— Column specification —————
Delimiter: ","
chr (5): IATA_CODE, AIRPORT, CITY, STATE, COUNTRY
dbl (2): LATITUDE, LONGITUDE

i Use `spec()` to retrieve the full column specification for this data.
i Specify the column types or set `show_col_types = FALSE` to quiet this message.

```

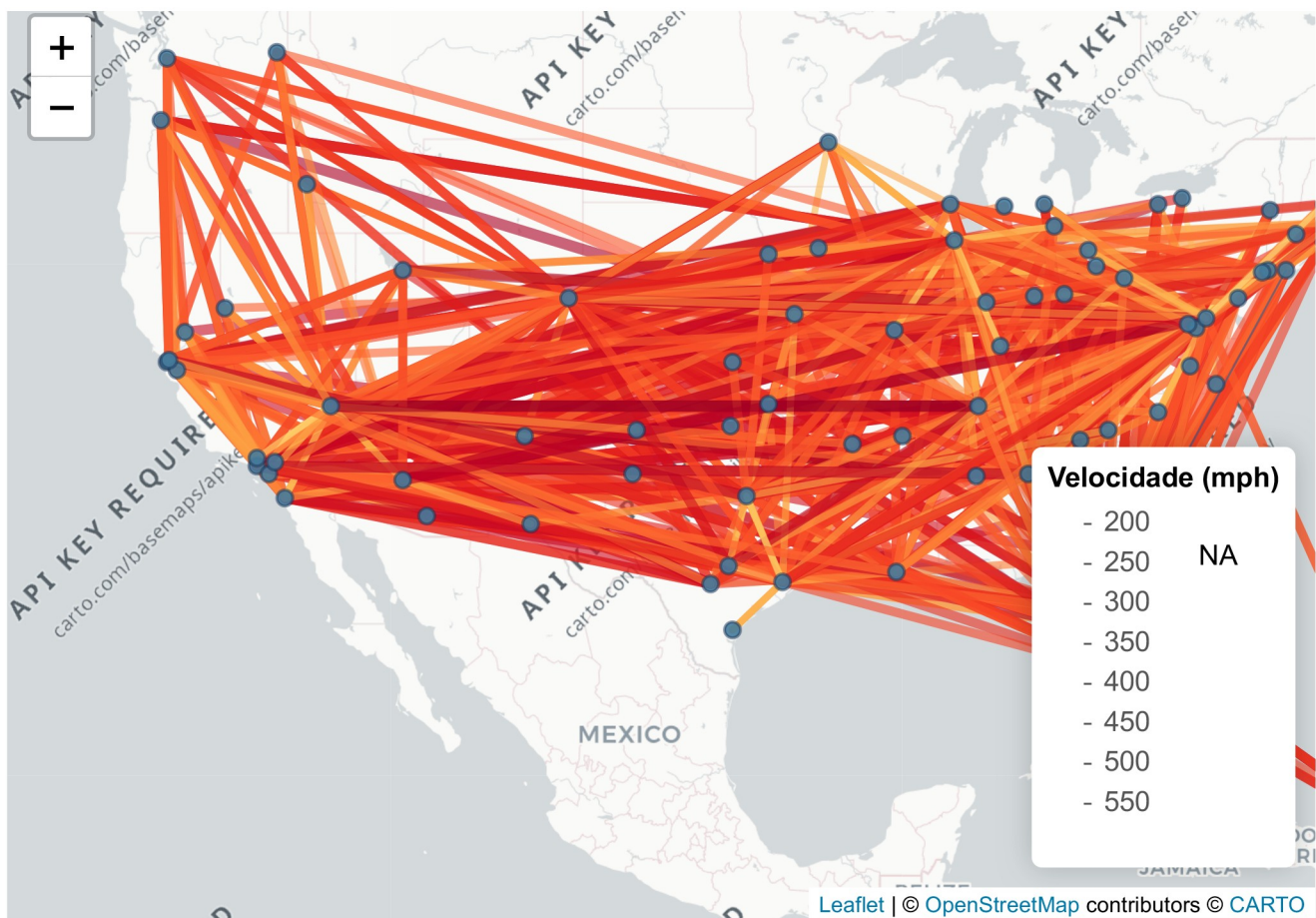
```
res$tabela
```

```

# A tibble: 1,772 × 31
  YEAR MONTH DAY DAY_OF_WEEK AIRLINE FLIGHT_NUMBER TAIL_NUMBER
  <dbl> <dbl> <dbl>      <dbl> <chr>      <dbl> <chr>
1  2015     1     1          4 WN          1762 N431WN
2  2015     1     1          4 WN          3564 N431WN
3  2015     1     1          4 WN          3564 N431WN
4  2015     1     1          4 WN          3564 N431WN
5  2015     1     2          5 WN          1793 N431WN
6  2015     1     2          5 WN          1793 N431WN
7  2015     1     2          5 WN          1793 N431WN
8  2015     1     2          5 WN          1793 N431WN
9  2015     1     2          5 WN          1793 N431WN
10 2015     1     2          5 WN          3144 N431WN
# i 1,762 more rows
# i 24 more variables: ORIGIN_AIRPORT <chr>, DESTINATION_AIRPORT <chr>,
#   SCHEDULED_DEPARTURE <chr>, DEPARTURE_TIME <chr>, DEPARTURE_DELAY <dbl>,
#   TAXI_OUT <dbl>, WHEELS_OFF <chr>, SCHEDULED_TIME <dbl>, ELAPSED_TIME <dbl>,
#   AIR_TIME <dbl>, DISTANCE <dbl>, WHEELS_ON <chr>, TAXI_IN <dbl>,
#   SCHEDULED_ARRIVAL <chr>, ARRIVAL_TIME <chr>, ARRIVAL_DELAY <dbl>,
#   DIVERTED <dbl>, CANCELLED <dbl>, CANCELLATION_REASON <chr>, ...

```

```
res$mapa
```



```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-09-16 20:34:55 -03"
```